Informe de Reflexión Individual sobre las Decisiones Grupales (Tarea 2)

1. Introducción y Contexto del Debate

Tras finalizar la fase individual de la Tarea 2, realizamos la puesta en común y análisis técnico entre las tres versiones desarrolladas por el equipo.

El objetivo de la discusión fue seleccionar la implementación más sólida y rigurosa para integrar como versión final en la rama `master`. Tras la evaluación técnica, defendí y propuse la elección de mi código por ser el único que cumple el 100% de las restricciones del enunciado y garantiza la máxima puntuación.

2. Defensa Argumentada de Mi Código

- **Cumplimiento estricto de restricciones docentes (Sin funciones Lambda):** En la Subtarea 7 (`hayColisionConAlguna` y `contarCeldasQueCumplen`), todas las combinaciones con `foldr` y `foldl` las definí mediante funciones auxiliares con nombre propio en bloques `where` (`comprueba`, `sumarFila`, `sumarCelda`), respetando la prohibición explícita del uso de lambdas.
- **Justificaciones escritas requeridas:** Incluye los comentarios explicativos de 2-3 líneas exigidos encima de las funciones de la Subtarea 7, argumentando la elección de `foldr` (por evaluación perezosa y cortocircuito con `||`) y `foldl` (por acumulación numérica estricta).
- **Recursión explícita y tipos correctos:**
    - `posicionesMetaRec` y `posicionesEnemigosRec` están implementadas con recursión explícita de doble nivel puro.
    - `mapPropio` y `filterPropio` están programadas con recursión explícita y mantienen el polimorfismo completo `(a -> b) -> [a] -> [b]`.

3. Análisis Técnico de las Alternativas Descartadas

Código de Javier Copete

- **Procesamiento incompleto en `prepararLineas`:** Su función auxiliar solo elimina espacios en blanco simples `' '`, fallando si la línea comienza con tabuladores (`\t`) o contiene espacios al final.
- **Menor eficiencia:** En `agruparPorCaracter` hace un doble recorrido al usar `takeWhile` y `dropWhile` por separado sobre el mismo tramo.

Código de Irene

- **Incumplimiento de normas (Uso de Lambdas y Comprensiones):** Emplea expresiones lambda directas (`\x acc -> ...`), prohibidas en el enunciado. Además, define `mapPropio` y `filterPropio` con comprensiones de listas (`[f x | x <- xs]`) en lugar de recursión explícita, y restringe el tipo de `mapPropio` a `(a -> a)`.
- **Error en `posicionesMetaRec`:** Su función auxiliar devuelve una sola tupla en lugar de una lista, por lo que ignora y pierde metas o enemigos si hay más de uno en la misma fila.
- **Bucle infinito:** En `agruparPorCaracter`, la llamada recursiva no reduce la lista al comparar caracteres iguales, colapsando la ejecución por recursión infinita.

4. Conclusión Final

El equipo aceptó por consenso mi propuesta de descartar las versiones de Irene (por errores de ejecución y vulneración de reglas docentes) y de Javier Copete (por falta de justificaciones requeridas y casos borde no cubiertos en cadenas).

Por tanto, acordamos seleccionar mi código como la versión oficial del grupo para fusionar en la rama `master`, asegurando una solución correcta, eficiente, con el bonus de QuickCheck completo y adaptada a los estándares de la asignatura.